opentide

workshop · 90 minutes · opentide 0.16.0

Detection engineering with a coding agent.

Ninety minutes, hands on: from an empty folder to your own operational detection platform, with one threat-aligned, validated rule in a pull request. You bring the intent, the agent does the typing, the engine checks, you approve.



What to bring. Check it now.

Everything runs on your laptop. The agent is the fast path, not the requirement.

✓ required

- Python 3.10 or later, and the rights to install packages.
  `a laptop where pip install is allowed`
- git, and enough familiarity to push a branch.
  `branch · commit · pull request`
- An IDE that reads JSON Schema, for YAML completion.
  `VS Code · Kiro · any schema-aware editor`
- Network egress. Test your proxy before the session.
  `pypi.org · github.com · opentide.org`

bring it if you want the full loop

- An agent: any harness, any model. Claude Code, Cursor, Codex, Kiro, VS Code with Copilot, Continue with a local model.
  `MCP support is what wires the opentide tools in`
- A git account, to push a branch and watch CI run.
  `github or gitlab · SaaS or self-managed`
- Platform or MISP credentials, for a real deploy.
  `your tenant, never ours · a dry run needs nothing`
- A threat report you actually care about.
  `otherwise we hand you one`

no agent The workshop still works. Every prompt in this deck maps to a CLI command and the slides show both, so you can type what the agent would have typed. Or pair up with someone who brought one.

what you leave with

One detection, end to end, built by your agent.

- a detection repo your agent scaffolded: Sentinel, GitHub CI, hooks
- the opentide MCP server and skills wired into your agent
- a threat → objective → rule chain that passes `validate --strict`
- a dry-run deploy plan and a pull request CI re-checks
- a prompt playbook you can reuse on Monday

01 **Set up**
one prompt · repo, mcp, skills, house rules

02 **Spec first**
threat → objective → rule, in that order

03 **Close the loop**
validate · fix · coverage · dry-run

04 **Ship it**
pull request · ci gates · promotion

05 **Your playbook**
prompts to keep · patterns · labs

Four labs. Seventy-five minutes.

Pair up: one person prompts, one person reviews. Swap at every lab.

lab 01 15 min

Set up

1. prompt 01: scaffold the repo
2. restart the agent, prompt 02
3. prompt 03: house rules

done when

validate --strict exits 0 and the agent lists the opentide tools

lab 02 30 min

First chain

1. pick an advisory
2. prompt 04: the threat
3. prompt 05: choose the signals
4. prompt 06: the rule

done when

a threat, an objective and a rule, linked by UUID, all green

lab 03 10 min

Break it

1. set an impact to 'Data Leak'
2. point detection_model at a UUID that does not exist
3. prompt: 'fix the build'

done when

green again, with no new UUIDs and no invented objects

lab 04 20 min

Ship it

1. prompt 07: coverage
2. prompt 08: preview the deploy
3. prompt 09: the pull request
4. prompt 10: the artefacts

done when

a pull request with the chain in its description, Validate job green

the clock **00:00** set up — **00:15** first chain — **00:45** break it — **00:55** ship it — **01:15** debrief

Three roles. Keep them separate.

The workshop works because nobody does someone else's job.

You

`intent and judgment`

- bring the intel and pick the platform
- choose which signals are worth building
- review the diff, approve promotion

The agent

`drafts and runs`

- searches the catalogue before writing
- drafts YAML from the templates
- runs the CLI, fixes what the engine flags

opentide

`the spec and the gates`

- ontology and taxonomy: schemas, vocabularies
- validation, chaining, coverage
- deploy plans, CI pipelines, docs

`no agent` You take the first two columns: you draft the YAML and run the CLI yourself. The third column does not change, and neither does a single gate in it.

01 / 05

Set up.

One prompt turns an empty folder into a detection repo your agent understands.



Let the agent scaffold the repo.

prompt 01 · scaffold — you → agent

Install `opentide==0.16.0` in a virtualenv.
Create an empty folder `Hacklu2026_ws` do `git init`and by using only opentide CLI in this empty folder,
- set up an opentide detection repository called "SOC Detections" for Example Corp.
- Configure Microsoft Sentinel and Splunk as platforms and GitHub Actions.
- Wire the opentide MCP server and skills for Claude Code and Generic agent
add the pre-commit hook, then run `generate` and `validate --strict`. Use non-interactive flags.

expect The interactive wizard needs a terminal. Agents pass explicit flags and `--yes`; without them opentide fails fast instead of waiting for input.

what the agent runs · real output

```
$ pip install 'opentide==0.16.0'
$ opentide setup --yes --name "SOC Detections" \
    --org "Example Corp" --platform sentinel --ci github
OK Setup complete
$ opentide setup mcp --claude-code --yes
OK MCP configuration generated
$ opentide setup skills --yes --claude-code \
    --name "SOC Detections" --org "Example Corp"
OK Agent skills installed
$ opentide setup hooks --yes
OK Validate-on-commit hooks configured
$ opentide generate
OK Full generation pipeline completed
$ opentide validate --strict
OK Validation passed, but no detection objects were found
```

What the agent just built.

A repository the engine, the CI and the agent all read the same way.

`objects/threats/`	threat vectors: what you defend against
`objects/objectives/`	detection objectives: what you must see
`objects/rules/`	platform rules: how a platform sees it
`.opentide/templates/`	object templates the agent drafts from
`.opentide/schemas/`	JSON schemas validate checks against
`.opentide/configurations/platforms/sentinel.toml`	platform config and tenants
`.github/workflows/opentide.yml`	validate → generate → deploy
`.pre-commit-config.yaml`	validate --strict on every commit
`.mcp.json`	the opentide MCP server, for Claude Code
`CLAUDE.md · .claude/skills/`	house rules and opentide skills

Restart the agent. Check its tools.

.mcp.json — written by setup mcp

```
{
  "mcpServers": {
    "opentide": {
      "command": "opentide",
      "args": ["mcp", "start"],
      "env": {
        "OPENTIDE_REPO_ROOT": "${CLAUDE_PROJECT_DIR}"
      }
    }
  }
}
```

prompt 02 · sanity check — you → agent

List the opentide MCP tools and skills you can use, and say which ones you would use to add a detection.

the opentide MCP tools

`search()`	objects by text, technique, actor, platform
`get_chaining()`	threat → objective → rule links
`coverage()`	rules per ATT&CK technique
`validation_report()`	structured issues to act on
`validate_rule()`	one rule, by UUID
`validate_query()`	tenant check, with live: true
`deploy_rule()`	deploy plan, dry_run=true
`deployment_status()`	where a rule is deployed
`skills`	two by default, more with `--install`

Add Prime Architect too.

A second MCP server next to opentide. I hand out accounts; you make your own token.

.mcp.json — add next to opentide

```
{
  "mcpServers": {
    "soc-prime-architect": {
      "url": "https://pubmcps.socprime.com/mcp",
      "headers": {
        "Authorization": "Bearer ${SOC_PRIME_MCP_TOKEN}",
        "Accept": "text/event-stream"
      }
    }
  }
}
```

sign in	tdm.socprime.com
token	Platform Settings → MCP Tokens
auth	Bearer token only

prime architect MCP tools

`deep_threat_research`	a report → summary, flow, rules
`generate_attack_flow`	ATT&CK techniques + a diagram
`generate_behavior_rule`	Sigma rules from the report
`translate`	between platforms, incl. opentide-mdr
`optimize_query`	recommendations for one SIEM
`generate_decision_tree`	analyst triage path for a rule
`search_skills · get_skill`	the agentic skills catalogue
`run_architect`	run one skill, by id

watch for Prime Architect drafts. The gate does not move: a translated rule still has to pass `validate --strict`.

Write the house rules once.

CLAUDE.md (or AGENTS.md) is read at the start of every session. Put the gates there, not in every prompt.

prompt 03 · house rules — you → agent

Add a "Detection workflow" section to the relevant file: AGENTS.md or CLAUDE.md . Threats, then objectives, then rules. Never invent vocabulary values or UUIDs. Nothing is done until validate --strict passes. Rules start in STAGING deploys are dry-run only, and you never touch credentials or promote to PRODUCTION.

tip `setup skills` also writes CLAUDE.md. Pass `--name` and `--org`, or the header falls back to the folder name.

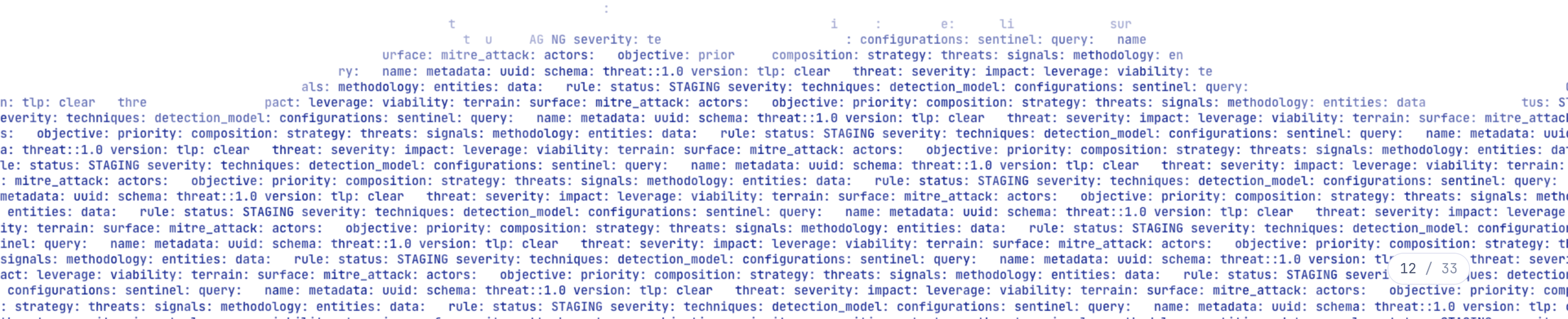
CLAUDE.md — the section it adds

```
## Detection workflow
- Order: threat → objective → rule, linked by UUID.
- Vocabulary values come from .opentide/schemas,
  never from memory.
- New UUIDs come from uuid4. Never edit existing ones.
- Done means validation_report is clean and
  `opentide validate --strict` exits 0.
- New rules start in STAGING.
- Deploys are dry_run=true. A human promotes.
- Never edit .opentide/configurations/ credentials.
```

02 / 05

Spec first.

Threat, then objective, then rule. The agent drafts each one; the ontology and the taxonomy keep them aligned.



Three objects. Three questions. Ask in this order.

A rule without an objective is a query with no reason to exist. Make the agent earn each step.



next Prompt 04 writes the threat, 05 the objective, 06 the rule. Each one starts from the UUID the previous one produced.

From intel to a threat object.

prompt 04 · threat — you → agent

Here is the advisory: `[paste the text or the URL]`. Using the opentide-threat-vector skill, create one threat object under `objects/threats/`. Map the ATT&CK techniques the text supports, and take impact, leverage, surface and actors from the opentide vocabularies only. Validate it and fix anything the engine reports.

watch for Techniques the advisory never mentions. Ask the agent to quote the sentence behind each one.

what to expect

- ✓ It searches first, so it does not duplicate a threat you already have.
  `search(query="LSASS")`
- ✓ It drafts from the generated template, not from memory.
  `.opentide/templates/threat.1.0.template.yaml`
- ✓ A fresh UUID, plus tlp, author and organisation.
  `python -c "import uuid; print(uuid.uuid4())"`
- ✓ It validates and loops until the report is clean.
  `validation_report(file=…)`

Ask for the plan before the YAML.

prompt 05 · objective — you → agent

Design a detection objective for that threat. Before writing any file, propose the signals: for each one the methodology, the entities, the data it needs, and whether Sentinel has that data. Wait for my OK, then write it under `objects/objectives/` and link the threat by UUID.

what comes back first

signal	method	entities	data
LSASS process access	Behavioural	Process, Hostname	have it
Dump file written	Behavioural	File, Hostname	have it
~~**Web shell spawn**~~	Behavioural	Process, URL	no data

you · keep the first two, log the third as a data gap

why The signals decide what gets built. This is where your judgment matters most, so make the agent stop and ask.

Then the rule, in STAGING

prompt 06 · rule — you → agent

Implement the "LSASS process access" signal as a Sentinel rule under `objects/rules/`, schema `rule::1.1`, status STAGING Set `detection_model` to the objective's UUID and take techniques from the threat. Schedule it hourly with a two-hour lookback. Validate, then show me the dry-run deploy plan.

expect The agent writes the KQL well. What it cannot tell you is whether the query returns data in your tenant: that check is yours, or CI's with `validate query --live`.

objects/rules/lsass-process-access.yaml — trimmed

```yaml
name: LSASS process access
metadata:
  uuid: c8f282eb-0286-…
  schema: rule::1.1
status: STAGING
severity: High
techniques: [T1003.001]
detection_model: f8817f95-c43e-…
configurations:
  sentinel:
    query: |
      DeviceEvents
      | where ActionType == "OpenProcessApiCall"
      | where FileName =~ "lsass.exe"
    scheduling:
      frequency: PT1H
      lookback: PT2H
```

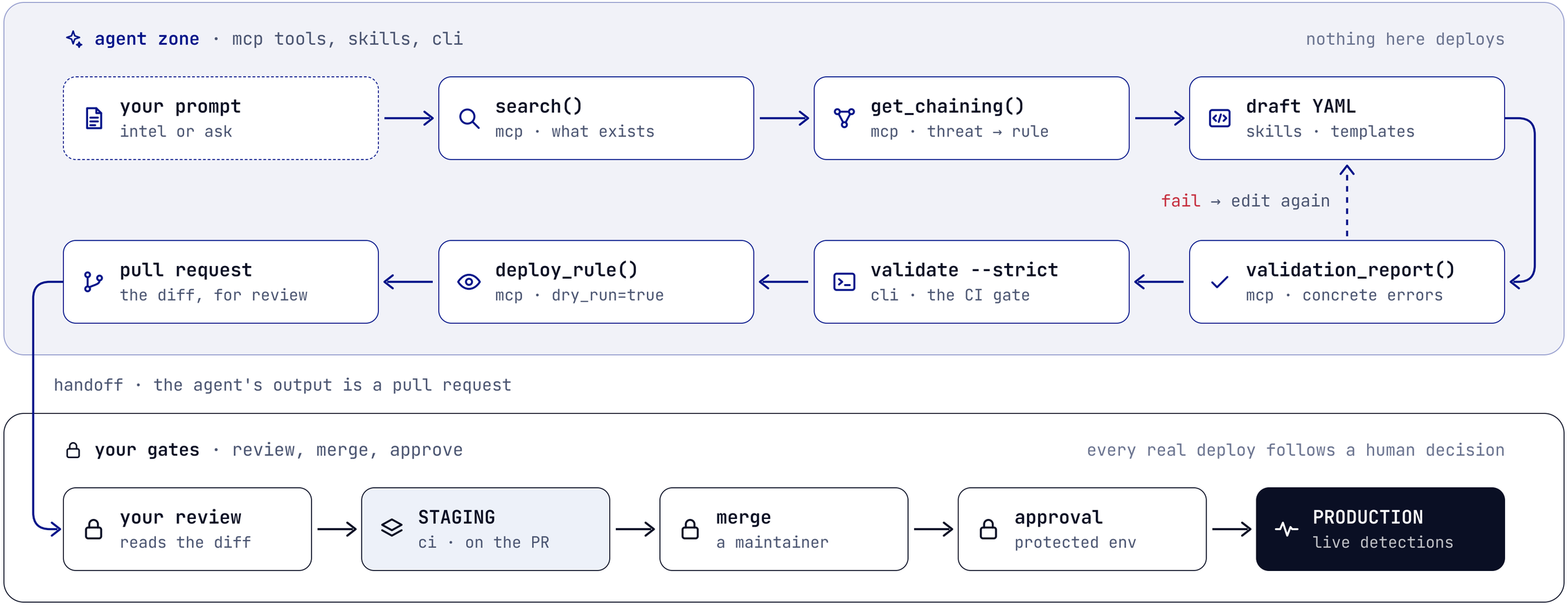
03 / 05

Close the loop.

The agent is done when the engine says so, not when the YAML exists.



The loop: the agent drafts, the engine checks, you promote.



When validation fails, the agent gets a fix list.

two mistakes an agent makes · real output

```
$ opentide validate
## objects/rules/sentinel-kql-rule.yaml
  [error] objects/rules/sentinel-kql-rule.yaml:15: detection_model:
    Unknown objective or signal reference
    '00000000-0000-4000-8002-deadbeef0000'
## objects/threats/simulated-actor.yaml
  [error] objects/threats/simulated-actor.yaml:15: threat.impact:
    Value 'Data Leak' is not a valid 'impact::1.0'
    vocabulary entry (suggestion: Data Breach)
FATAL: Validation failed          # exit 1
```

validation_report · one issue


```
{
  "code": "vocab_unknown",
  "severity": "error",
  "object_type": "threat",
  "yaml_line": 14,
  "field_path": ["threat", "impact"],
  "message": "Value 'Data Leak' is not a valid
              'impact::1.0' vocabulary entry",
  "suggestion": "Data Breach"
}
```


- ✓ Applies the vocabulary suggestion.
- ✓ Restores the real objective UUID with `search()`. It never invents an objective to satisfy the reference.

Ask the catalogue, not the model.

prompt 07 · coverage — you → agent

What do we cover for T1059 and T1003? For each technique, list the rules with their objectives and threats, then tell me which objectives still have no rule.

expect Coverage counts rules. Chaining tells you why each one exists. Ask for both, and answers come from the repository, not from the model's memory.

the calls behind the answer

`coverage(technique="T1059")` → 1 rule

`get_chaining(uuid=…8003…0001)` → rule → objective → threat

`search(type="objective")` → objectives with no rule

the same answer from the CLI — real output


```
{
  "coverage": {
    "technique": "T1059",
    "rules": ["00000000-0000-4000-8003-000000000001"],
    "count": 1
  }
}
```

Preview every deploy.

prompt 08 · preview — you → agent

Preview the Sentinel deployment for everything in STAGING. Do not deploy for real. Summarise what would change.

one STAGING rule, no tenant yet · real output

```
$ opentide deploy --platform sentinel --dry-run
== MDR Deployment ==
OK Deployment completed
  advice: add (or uncomment) a [[tenants]] entry in
  .opentide/configurations/platforms/sentinel.toml
WARNING sentinel has no tenants configured in
.opentide/configurations/platforms/sentinel.toml,
so a real deploy would stop
```

what to expect

- ✓ Through MCP it previews one rule at a time.
  `deploy_rule(uuid, platform="sentinel", dry_run=true)`
- ✓ The plan names what is missing. Tenants and credentials are yours to add, never the agent's.
  `[[tenants]]` · GitHub secrets
- ✓ With credentials, `validate query --live` checks the catalogue queries against your workspace.
  `opentide validate query --platform sentinel --live`

04 / 05

Ship it.

The agent's output is a pull request. CI runs every gate again; humans promote.



One chain, one pull request.

prompt 09 · pull request — you → agent

Create a branch, commit the threat, the objective and the rule, and open a pull request. In the description: the intel source, the chain with UUIDs, the signals we chose not to build and why, and the result of `opentide validate --strict`.

expect The pre-commit hook runs validate on the commit itself, so a broken object never reaches the branch.

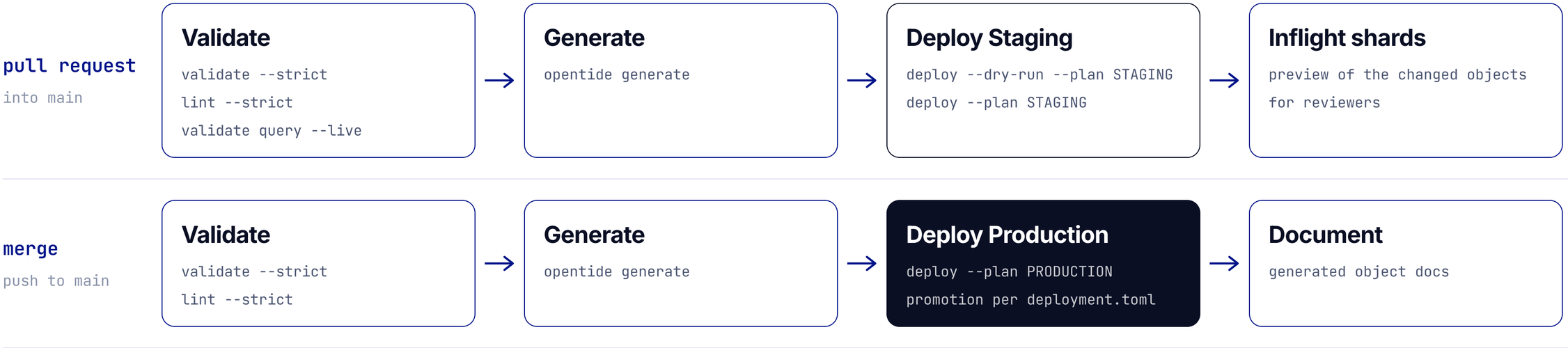
what lands in the PR description

```
detections: LSASS process access (T1003.001)

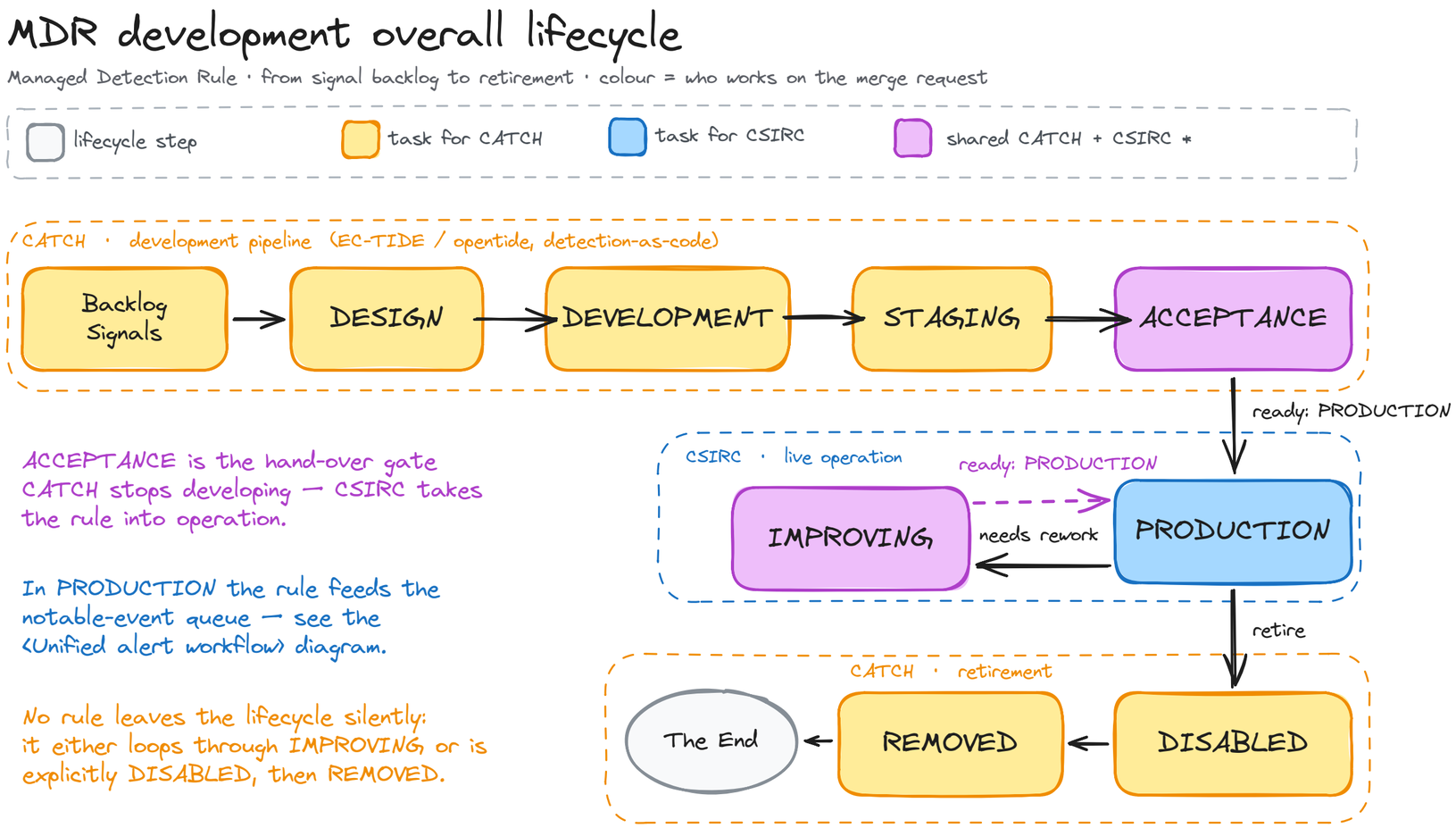
intel     advisory, ransomware crew dumping LSASS
chain     threat    aa36f6de-4cc8-…
          objective f8817f95-c43e-…
          rule      c8f282eb-0286-…  (STAGING)
skipped   web shell spawn: no web telemetry yet
gates     opentide validate --strict     exit 0
          deploy --platform sentinel --dry-run   ok
```

CI runs the gates the agent already ran.

Same engine, same version pin, no agent in the loop. A pass on the laptop is a pass in CI.



credentials come from GitHub secrets · tenant config is read from main, never from the pull request · --skip-unconfigured keeps a repo with no tenant green · put production behind a protected environment



gate Backlog to `PRODUCTION`. ACCEPTANCE is the hand-over gate: the alert handling manager reviews the rule before it goes live, and `IMPROVING` is the loop back when it needs rework. No rule leaves the lifecycle silently.

Documentation nobody has to write.

The markdown wiki, the explorer graph, the ATT&CK layer and the MISP event all fall out of the objects you just validated, so none of them can drift from what you run.

prompt 10 · artefacts — you → agent

Generate the documentation and the exports for this repository, then preview what sharing to MISP would publish. Show me the markdown page for our rule, the ATT&CK layer, and the share preview. Push nothing.

watch for Sharing is not deployment: a destination is a MISP instance, not a platform. `max_tlp` is a hard ceiling per destination, so a TLP:AMBER object cannot leave.

what gets generated

- A markdown page per threat, objective and rule, with Mermaid coverage and chaining diagrams, backlinked both ways.
  `opentide generate docs --changed`
- The whole catalogue as a graph you can browse, with a prebuilt search index and no backend to patch.
  `.opentide/exports/explorer.bundle.json`
- An ATT&CK layer: green is full coverage, red a threat with no rule, blue a rule with no threat behind it.
  `opentide generate exports navigator · layer 4.5`
- One MISP Event per object. Preview makes the same decision as push, with no request and no ledger write.
  `opentide share preview · sharing.toml`

What the agent does. What you keep.

Let the agent

- search, read and summarise objects and coverage
- draft new objects from the templates
- fix dangling references and vocabulary errors the validator flags
- preview deploys with `dry_run=true`
- open the pull request and describe it

Keep for yourself

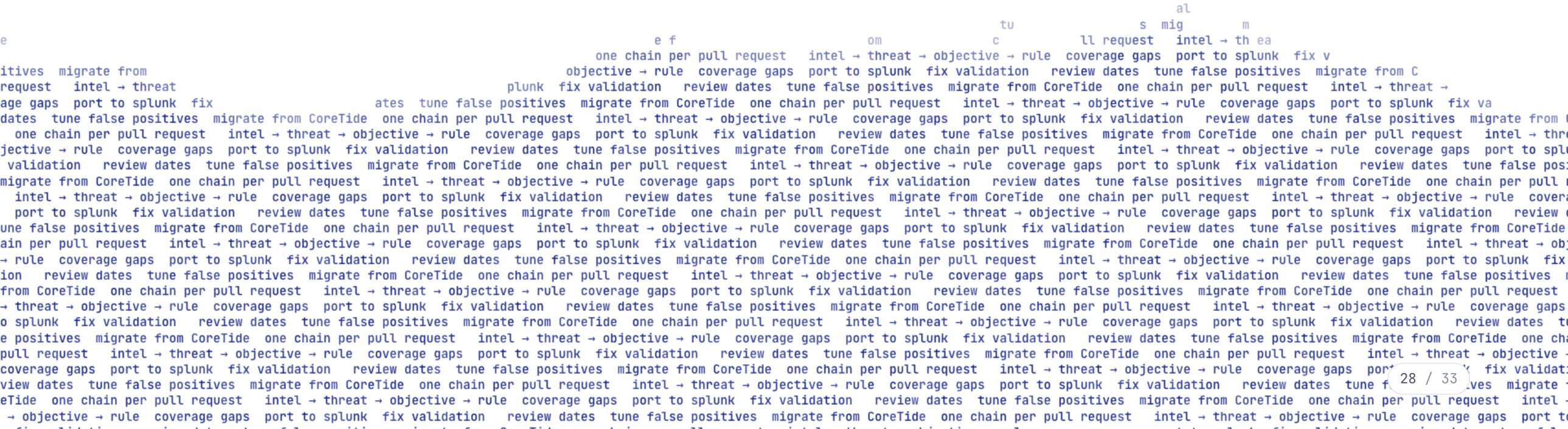
- real deploys, and any promotion to PRODUCTION
- credentials and plans in `.opentide/configurations/`
- deciding that a query works: look at results in the tenant
- resolving UUID conflicts; never regenerate UUIDs
- which signals are worth building

enforce Write the right-hand column into CLAUDE.md (prompt 03), and protect main and the production environment in GitHub.

05 / 05

Your playbook.

Prompts that work, what to expect from them, and four labs to run today.



Six prompts to keep.

Intel to detection

"Here is an advisory. Build the threat, propose objectives and signals, wait for my OK, then write the rules. One pull request."

`threat · objective · rules · one PR`

Coverage gaps

"Which ATT&CK techniques in our threats have no rule yet? Rank them by threat criticality."

`coverage() · get_chaining() · search()`

Port a rule

"Enable Splunk, then add a splunk configuration to rule <uuid> with the same logic in SPL."

`setup platforms --splunk · one new config block`

Fix the build

"CI failed. Read opentide --json validate and fix every issue. Do not touch any UUID."

`validation_report() until clean`

Review sweep

"List rules whose review date is missing or past. For each: summarise it and propose keep, tune or retire."

`rule::1.1 review dates · a triage list`

Leave CoreTide

"Migrate this CoreTide repository to opentide, following the migration prompt in the opentide docs."

`opentide.org/docs/usage/migration/prompt`

Prompt like a spec, not a wish.

✓ Do

- name the object and the order: threat, objective, rule
- give the intel, ask for the mapping, check the mapping
- ask for the signals before the objective is written
- end every prompt with the gate: `validate --strict` passes
- keep one chain per pull request

× Don't

- ask for "a rule for T1059" with no threat behind it
- let it pick vocabulary values or UUIDs from memory
- accept "done" without the validation output
- let it deploy for real or flip a status to PRODUCTION
- paste credentials into the chat

test If a prompt would not survive as a line in a pull request description, it is too vague for the agent too.

Four labs. Seventy-five minutes.

Pair up: one person prompts, one person reviews. Swap at every lab.

lab 01 15 min

Set up

1. prompt 01: scaffold the repo
2. restart the agent, prompt 02
3. prompt 03: house rules

done when

validate --strict exits 0 and the agent lists the opentide tools

lab 02 30 min

First chain

1. pick an advisory
2. prompt 04: the threat
3. prompt 05: choose the signals
4. prompt 06: the rule

done when

a threat, an objective and a rule, linked by UUID, all green

lab 03 10 min

Break it

1. set an impact to 'Data Leak'
2. point detection_model at a UUID that does not exist
3. prompt: 'fix the build'

done when

green again, with no new UUIDs and no invented objects

lab 04 20 min

Ship it

1. prompt 07: coverage
2. prompt 08: preview the deploy
3. prompt 09: the pull request
4. prompt 10: the artefacts

done when

a pull request with the chain in its description, Validate job green

the clock **00:00** set up — **00:15** first chain — **00:45** break it — **00:55** ship it — **01:15** debrief

Take it home.

The same five commands, then hand the keyboard to your agent.

a new detection repo — python 3.10+

```
$ pip install 'opentide==0.16.0'
$ opentide setup --yes --name "SOC Detections" \
    --platform sentinel --ci github
$ opentide setup mcp --claude-code --yes
$ opentide setup skills --yes --claude-code
$ opentide generate && opentide validate --strict
```

tutorial	opentide.org/docs/usage/tutorial
agents	…/usage/workflows/agentic-setup
mcp tools	opentide.org/docs/mcp/tools
skills	github.com/opentidehq/skills

scan

opentide.org

opentide

Agents write the queries. The spec keeps them aligned.

Threat first, plan before YAML, the gate before done, a human before production.

opentide.org github.com/opentidehq pip install opentide

